Project 3 Report: Pothole Tracking and Repair System (PTRS)

Overview

This database was designed for the Austin Department of Public Works to support the Pothole Tracking and Repair System (PTRS). The system allows citizens to report potholes, allows the department to assign work crews and equipment to repairs, and allows the city to record property damage claims connected to pothole incidents. The database stores data about citizens who report potholes, pothole reports, repair work orders, work crews, equipment used during repairs, and any related property damage claims.

Design Choices

The design follows the relational model and keeps all tables in BCNF. To maintain normalization, data that belongs to one entity is stored only once, while related records are connected through foreign keys:

- **Citizen information** is stored in `citizen_user`.
- **Districts** are stored in `district` and each pothole references one district.
- **Pothole reports** are stored in `pothole`.
- **Work crews** are stored in `work_crew`.
- **Equipment** is stored in `equipment`.
- **Work orders** are stored in `work_order`; each work order is tied to exactly one pothole, and a pothole may have zero or one work order.
- Because one work order can use multiple pieces of equipment and one piece of equipment can be used on many work orders, the many-to-many relationship is implemented with `work_order_equipment`.
- **Property damage claims** are stored in `property_damage_claim`, and each claim references both the pothole involved and the citizen who filed the claim.

This structure avoids repeating district names, citizen information, crew information, and equipment data across multiple operational records. It also allows the application software to retrieve complete repair and claim information through joins rather than by storing the same data in several places.

Assumptions

1. **District determination**: the application determines the district from the street address and stores the resulting `district_id` in the pothole table.
2. **Repair priority**: repair priority is automatically derived from pothole size using a generated column:
   - 1-4 = Low
   - 5-7 = Medium
   - 8-10 = High
3. **Cost of repair** is computed in queries rather than stored directly, based on labor, filler material, and equipment usage. This avoids storing a derivable value and keeps the design normalized.
4. The assignment description says a work order includes pothole size and location, but those attributes are already stored in `pothole`. To preserve BCNF, the work order references the pothole and the application can retrieve size and location by joining the two tables.

Table Purposes

`district`

`district` stores the city districts used for routing reports and summarizing repair activity. The primary key is `district_id`. The application uses this table when assigning each reported pothole to a city district and when generating reports grouped by district.

`citizen_user`

`citizen_user` stores each registered citizen who submits pothole reports or property damage claims. The primary key is `user_id`. This table is referenced by both `pothole` and `property_damage_claim` so the application can identify who reported a pothole and who filed a claim.

`pothole`

`pothole` stores each pothole report submitted through the PTRS application. The primary key is `pothole_id`, and the table contains foreign keys `user_id` and `district_id` to connect each report to the reporting citizen and the correct district. The application uses this table as the central operational record for location, severity, and repair-priority tracking.

`work_crew`

`work_crew` stores each repair crew maintained by the department. The primary key is `crew_id`, and the table records the crew name and the number of workers on the crew. The application uses this table when assigning crews to work orders and when generating workload reports by crew.

`equipment`

`equipment` stores each piece of repair equipment and its hourly operating rate. The primary key is `equipment_id`. The application uses this table to track equipment availability and to compute repair costs based on the equipment assigned to a work order.

`work_order`

`work_order` stores the repair activity associated with a pothole. The primary key is `work_order_id`, and the foreign keys `pothole_id` and `crew_id` connect each work order to one pothole and one assigned repair crew. The application uses this table to track scheduling, hours worked, repair status, filler material usage, and the inputs needed to calculate repair cost.

`work_order_equipment`

`work_order_equipment` is the bridge table between `work_order` and `equipment`. Its composite primary key is (`work_order_id`, `equipment_id`), and it stores the number of usage hours for each equipment assignment. The application uses this table to represent the many-to-many relationship between work orders and equipment and to support cost and utilization reporting.

`property_damage_claim`

`property_damage_claim` stores each property damage claim related to a pothole incident. The primary key is `claim_id`, and the foreign keys `pothole_id` and `user_id` connect the claim to the affected pothole and the citizen who filed it. The application uses this table to track claim type, claim amount, filing date, and review status for administrative reporting.

Relationships

- `citizen_user` to `pothole` is a one-to-many relationship: one citizen can report `0..*` potholes, and each pothole is reported by `1..1` citizen.
- `district` to `pothole` is a one-to-many relationship: one district can contain `0..*` potholes, and each pothole belongs to `1..1` district.
- `pothole` to `work_order` is a one-to-one relationship with optional participation on the pothole side: one pothole can have `0..1` work order, and each work order belongs to `1..1` pothole.
- `work_crew` to `work_order` is a one-to-many relationship: one work crew can be assigned `0..*` work orders, and each work order is assigned to `1..1` work crew.
- `work_order` to `equipment` is a many-to-many relationship that is resolved by `work_order_equipment`: one work order can connect to `0..*` equipment assignment rows, and one equipment item can connect to `0..*` assignment rows.
- `pothole` to `property_damage_claim` is a one-to-many relationship: one pothole can have `0..*` property damage claims, and each claim refers to `1..1` pothole.
- `citizen_user` to `property_damage_claim` is a one-to-many relationship: one citizen can file `0..*` claims, and each claim is filed by `1..1` citizen.

Application Usage

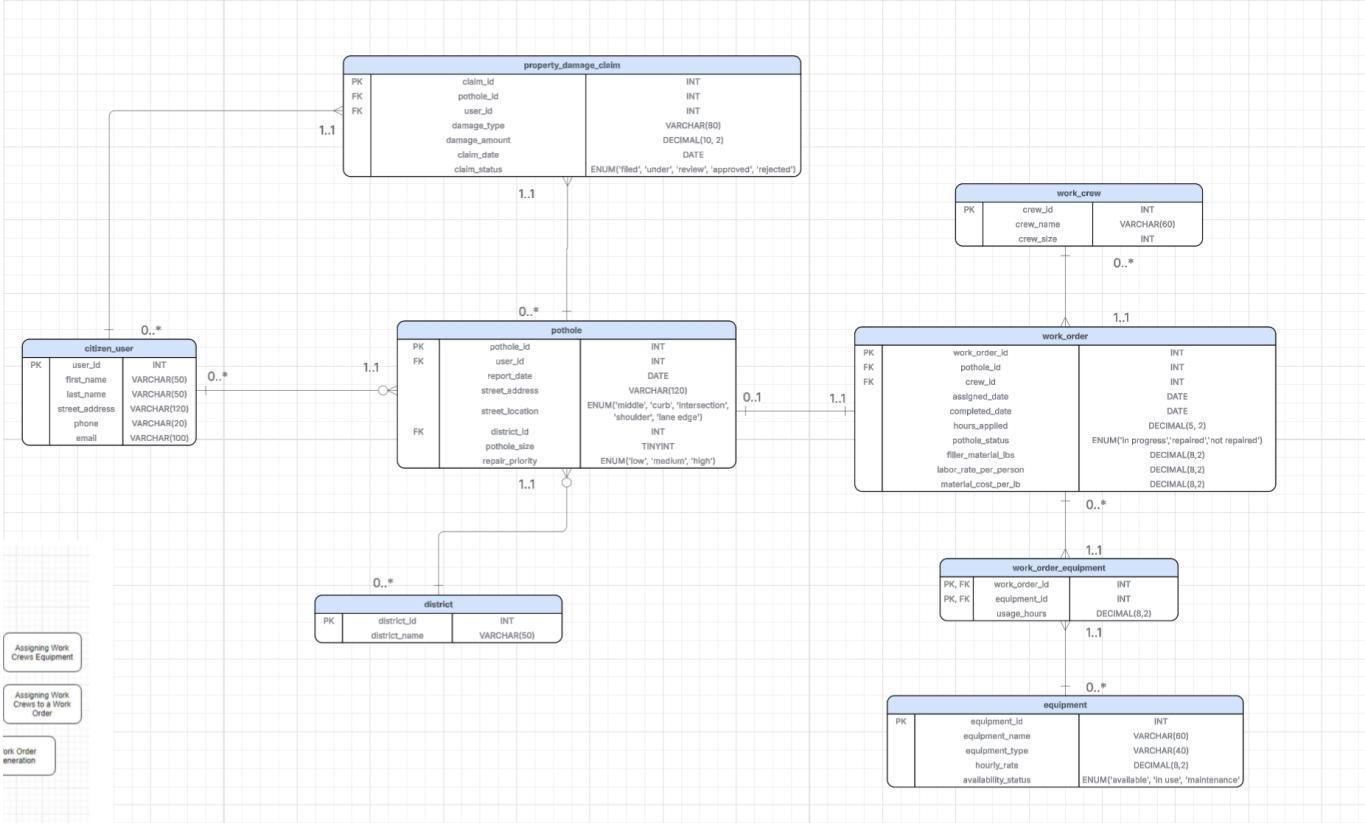
The application inserts citizen account information into `citizen_user`, accepts pothole submissions into `pothole`, determines the district from the submitted address, and derives repair priority from pothole size. Repair staff then create `work_order` records and assign crews and equipment through `work_crew`, `equipment`, and `work_order_equipment`. If a citizen reports damage caused by a pothole, the application records the claim in `property_damage_claim` and links it to both the citizen and the pothole.

Normalization and BCNF

The design keeps each table focused on a single subject so that non-key attributes depend only on the key, the whole key, and nothing but the key. District names are stored once in `district`, crew details are stored once in `work_crew`, and equipment details are stored once in `equipment`. The many-to-many relationship between work orders and equipment is handled by `work_order_equipment`, which avoids repeated equipment columns inside `work_order`. Property damage data is kept in its own table rather than being mixed into pothole or citizen records. These choices reduce redundancy, support BCNF, and prevent common update anomalies.

E-R Diagram

The finalized E-R diagram includes all table names, column names, data types, primary keys, foreign keys, and minimum/maximum cardinalities for each relationship.



E-R Diagram Cardinalities

Suggested cardinalities:

- `citizen_user` `0..*` to `pothole` and `pothole` `1..1` to `citizen_user`
- `district` `0..*` to `pothole` and `pothole` `1..1` to `district`
- `pothole` `0..1` to `work_order` and `work_order` `1..1` to `pothole`
- `work_crew` `0..*` to `work_order` and `work_order` `1..1` to `work_crew`
- `work_order` `0..*` to `work_order_equipment` and `work_order_equipment` `1..1` to `work_order`
- `equipment` `0..*` to `work_order_equipment` and `work_order_equipment` `1..1` to `equipment`
- `pothole` `0..*` to `property_damage_claim` and `property_damage_claim` `1..1` to `pothole`
- `citizen_user` `0..*` to `property_damage_claim` and `property_damage_claim` `1..1` to `citizen_user`